Machine Translated by Google

# Capítulo 3: Paralelização

## Visão geral do padrão de paralelização

Nos capítulos anteriores, exploramos o encadeamento de prompts para fluxos de trabalho sequenciais e o roteamento para tomada de decisões dinâmicas e transições entre diferentes caminhos. Embora esses padrões sejam essenciais, muitas tarefas complexas de agentes envolvem múltiplas subtarefas que podem ser executadas ***simultaneamente,*** em vez de uma após a outra. É aqui que o padrão **de Paralelização** se torna crucial.

A paralelização envolve a execução simultânea de múltiplos componentes, como chamadas LLM, usos de ferramentas ou até mesmo subagentes inteiros (ver Fig. 1). Em vez de esperar que uma etapa seja concluída antes de iniciar a próxima, a execução paralela permite que tarefas independentes sejam executadas ao mesmo tempo, reduzindo significativamente o tempo total de execução para tarefas que podem ser divididas em partes independentes.

Considere um agente projetado para pesquisar um tópico e resumir suas descobertas. Uma abordagem sequencial poderia:

1. Procure a Fonte A.
2. Resuma a Fonte A.
3. Procure a Fonte B.
4. Resuma a Fonte B.
5. Elabore uma resposta final a partir dos resumos A e B.

Uma abordagem paralela poderia, em vez disso:

1. Pesquise simultaneamente a Fonte A ***e*** a Fonte B.
2. Após a conclusão de ambas as pesquisas, resuma a Fonte A ***e*** resuma a Fonte B. simultaneamente.
3. Sintetize uma resposta final a partir dos resumos A e B (esta etapa é normalmente sequencial, aguardando a conclusão das etapas paralelas).

A ideia central é identificar as partes do fluxo de trabalho que não dependem da saída de outras partes e executá-las em paralelo. Isso é particularmente eficaz ao lidar com serviços externos (como APIs ou bancos de dados) que apresentam latência, pois permite o envio de várias solicitações simultaneamente.

A implementação da paralelização geralmente requer frameworks que suportem execução assíncrona ou multithreading/multiprocessamento. Frameworks agenticos modernos são

Machine Translated by Google

Projetado com operações assíncronas em mente, permitindo que você defina facilmente etapas que podem ser executadas em paralelo.

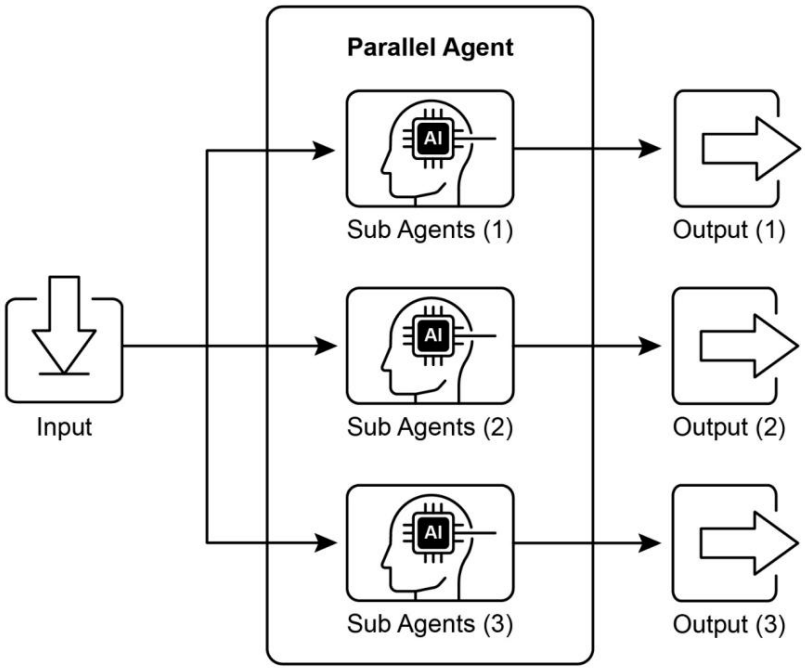


Figura 1. Exemplo de paralelização com subagentes

Frameworks como LangChain, LangGraph e Google ADK fornecem mecanismos para execução paralela. Na Linguagem de Expressão LangChain (LCEL), você pode obter execução paralela combinando objetos executáveis usando operadores como | (para sequencial) e estruturando suas cadeias ou grafos para terem ramificações que são executadas simultaneamente.

O LangGraph, com sua estrutura de grafo, permite definir múltiplos nós que podem ser executados a partir de uma única transição de estado, possibilitando ramificações paralelas no fluxo de trabalho. O Google ADK fornece mecanismos nativos robustos para facilitar e gerenciar a execução paralela de agentes, aprimorando significativamente a eficiência e a escalabilidade de sistemas multiagentes complexos. Essa capacidade inerente ao framework ADK permite que os desenvolvedores projetem e implementem soluções onde múltiplos agentes podem operar simultaneamente, em vez de sequencialmente.

Machine Translated by Google

O padrão de paralelização é vital para melhorar a eficiência e a capacidade de resposta de sistemas de agentes, especialmente ao lidar com tarefas que envolvem múltiplas buscas independentes, cálculos ou interações com serviços externos. É uma técnica fundamental para otimizar o desempenho de fluxos de trabalho complexos de agentes.

# Aplicações práticas e casos de uso

A paralelização é um padrão poderoso para otimizar o desempenho de agentes em diversas aplicações:

1. Coleta e Pesquisa de Informações: Coletar informações de múltiplas fontes simultaneamente é um caso de uso clássico. • **Caso de Uso:** Um agente pesquisando uma empresa. • **Tarefas Paralelas:** Pesquisar artigos de notícias, obter dados de ações, verificar menções em mídias sociais e consultar um banco de dados da empresa, tudo ao mesmo tempo. • **Benefício:** Obtém uma visão abrangente muito mais rapidamente do que pesquisas sequenciais.

2. Processamento e Análise de Dados: Aplicação de diferentes técnicas de análise ou processamento simultâneo de diferentes segmentos de dados. • **Caso de Uso:** Um agente analisando o feedback do cliente. • **Tarefas Paralelas:** Executar análise de sentimento, extrair palavras-chave, categorizar o feedback e identificar problemas urgentes simultaneamente em um lote de entradas de feedback.

• **Benefício:** Fornece uma análise multifacetada rapidamente.

3. Interação com múltiplas APIs ou ferramentas: Chamada de múltiplas APIs ou ferramentas independentes para coletar diferentes tipos de informações ou executar diferentes ações. • **Caso de uso:** Um agente de planejamento de viagens.

• **Tarefas paralelas:** Verificar preços de voos, pesquisar disponibilidade de hotéis, consultar informações locais eventos e encontre recomendações de restaurantes simultaneamente.

• **Benefício:** Apresenta um plano de viagem completo mais rapidamente.

4. Geração de conteúdo com múltiplos componentes: Geração paralela de diferentes partes de um conteúdo complexo. • **Caso de uso:** Um agente criando um e-mail de marketing.

• **Tarefas paralelas:** gerar uma linha de assunto, redigir o corpo do e-mail, encontrar uma imagem relevante e criar um texto de botão de chamada para ação simultaneamente.

• **Benefício:** Permite montar o e-mail final de forma mais eficiente.

5. Validação e Verificação:

Machine Translated by Google

Executar várias verificações ou validações independentes simultaneamente. • **Caso de uso:** Um agente verificando a entrada do usuário. •

**Tarefas paralelas:** Verificar o formato do e-mail, validar o número de telefone, verificar o endereço em um banco de dados e verificar a presença de palavrões simultaneamente.

• **Benefício:** Fornece feedback mais rápido sobre a validade da entrada de dados.

6. Processamento Multimodal:

Processamento simultâneo de diferentes modalidades (texto, imagem, áudio) da mesma entrada. • **Caso de uso:** Um agente analisando uma publicação em uma rede social contendo texto e imagem.

• **Tarefas paralelas:** Analise o texto em busca de sentimentos e palavras-chave ***e*** analise a imagem em busca de objetos e descrição da cena simultaneamente. •

**Benefício:** Integra insights de diferentes modalidades mais rapidamente.

7. Teste A/B ou Geração de Múltiplas Opções: Gerar múltiplas variações de uma resposta ou saída em paralelo para selecionar a melhor. um.

• **Caso de uso:** Um agente que gera diferentes opções de texto criativo.

• **Tarefas paralelas:** Gere três títulos diferentes para um artigo simultaneamente, usando instruções ou modelos ligeiramente diferentes. • **Benefício:** Permite uma comparação rápida e a seleção da melhor opção.

A paralelização é uma técnica fundamental de otimização no design de agentes, permitindo que os desenvolvedores criem aplicativos mais eficientes e responsivos, aproveitando a execução concorrente de tarefas independentes.

# Exemplo prático de código (LangChain)

A execução paralela dentro da estrutura LangChain é facilitada pela Linguagem de Expressão LangChain (LCEL). O método principal envolve estruturar múltiplos componentes executáveis dentro de um dicionário ou lista. Quando essa coleção é passada como entrada para um componente subsequente na cadeia, o ambiente de execução LCEL executa os componentes executáveis contidos simultaneamente.

No contexto do LangGraph, esse princípio é aplicado à topologia do grafo. Fluxos de trabalho paralelos são definidos arquitetando o grafo de forma que múltiplos nós, sem dependências sequenciais diretas, possam ser iniciados a partir de um único nó comum. Esses caminhos paralelos são executados independentemente antes que seus resultados possam ser agregados em um ponto de convergência subsequente no grafo.

A implementação a seguir demonstra um fluxo de trabalho de processamento paralelo construído com o framework LangChain. Este fluxo de trabalho foi projetado para executar duas tarefas.

Machine Translated by Google

operações independentes executadas simultaneamente em resposta a uma única consulta do usuário. Esses processos paralelos são instanciados como cadeias ou funções distintas, e suas respectivas saídas são posteriormente agregadas em um resultado unificado.

Os pré-requisitos para esta implementação incluem a instalação dos pacotes Python necessários, como langchain, langchain-community e uma biblioteca provedora de modelos como langchain-openai. Além disso, uma chave de API válida para o modelo de linguagem escolhido deve ser configurada no ambiente local para autenticação.

```
import os
import asyncio from
typing import Optional

from langchain_openai import ChatOpenAI from
langchain_core.prompts import ChatPromptTemplate from
langchain_core.output_parsers import StrOutputParser from langchain_core.runnables
import Runnable, RunnableParallel, RunnablePassthrough


# --- Configuração --- # Certifique-se
de que a variável de ambiente da sua chave de API esteja definida (por exemplo,
OPENAI_API_KEY)
tente:
    llm: Optional[ChatOpenAI] = ChatOpenAI(model="gpt-4o-mini", temperature=0.7)


exceto Exception como e:
    print(f"Erro ao inicializar o modelo de linguagem: {e}")
    llm = Nenhum

# --- Definir Cadeias Independentes --- # Essas três
cadeias representam tarefas distintas que podem ser executadas em paralelo.


summarize_chain: Runnable =
    ( ChatPromptTemplate.from_messages([
            ("sistema", "Resuma o seguinte tópico de forma concisa:"), ("usuário", "{tópico}")

    ])
     | llm
     | StrOutputParser()
)

questions_chain: Runnable =
    ( ChatPromptTemplate.from_messages([
```

Machine Translated by Google

```
                ("sistema", "Gere três perguntas interessantes sobre o
seguinte tópico:"),
                ("usuário", "{tópico}")
        ])
        | llm
        | StrOutputParser()
)

terms_chain: Runnable =
        ( ChatPromptTemplate.from_messages([ ("system",
                "Identifique de 5 a 10 termos-chave do seguinte tópico, separados por vírgulas:"), ("user", "{topic}")



        ])
        | llm
        | StrOutputParser()
)

# --- Construir a cadeia paralela + síntese ---

# 1. Defina o bloco de tarefas a serem executadas em paralelo. Os resultados disso, # map_chain =

            Juntamente com o tópico original, será encaminhado para a próxima etapa.
RunnableParallel(
        {
                "summary": summarize_chain, "questions":
                questions_chain, "key_terms": terms_chain,
                "topic": RunnablePassthrough(), #
                Passa o tópico original }



)

# 2. Defina o prompt de síntese final que combinará os resultados paralelos. synthesis_prompt =

ChatPromptTemplate.from_messages([ ("system", """Com base nas seguintes
        informações: Resumo: {summary}

          Perguntas relacionadas: {questions}
          Termos-chave: {key_terms}
          Sintetize uma resposta abrangente."""), ("usuário", "Tópico
        original: {tópico}")
])

# 3. Construa a cadeia completa encadeando os resultados paralelos diretamente #
            no prompt de síntese, seguido pelo LLM e saída.
analisador sintático.
```

Machine Translated by Google

```
cadeia_paralela_completa = cadeia_de_mapeamento | prompt_de_síntese | llm |
StrOutputParser()

# --- Executar a cadeia --- async def
run_parallel_example(topic: str) -> None:
    """

    Invoca de forma assíncrona a cadeia de processamento paralelo com um tópico específico e
imprime o resultado
    sintetizado.



    Argumentos: tópico: O tópico de entrada a ser processado pelo LangChain
correntes.
    """

    se não llm:
        print("LLM não inicializado. Não é possível executar o exemplo.")
        retornar


    print(f"\n--- Executando Exemplo de LangChain Paralelo para o Tópico: '{topic}' ---") try: # A
 entrada para `ainvoke`
    é a
        string única 'topic', # então passada para cada runnable em `map_chain`. response =
            await full_parallel_chain.ainvoke(topic) print("\n--- Resposta Final ---")
            print(response) except Exception as e:




            print(f"\nOcorreu um erro durante a execução da cadeia: {e}")

Se __name__ == "__main__": test_topic
    = "A história da exploração espacial"
    Em Python 3.7+, `asyncio.run` é a maneira padrão de executar uma função assíncrona.
função.
    asyncio.run(run_parallel_example(test_topic))
```

O código Python fornecido implementa uma aplicação LangChain projetada para processar um tópico específico de forma eficiente, aproveitando a execução paralela. Observe que o asyncio oferece concorrência, não paralelismo. Ele consegue isso em uma única thread usando um loop de eventos que alterna inteligentemente entre tarefas quando uma delas está ociosa (por exemplo, aguardando uma requisição de rede). Isso cria o efeito de múltiplas tarefas progredindo simultaneamente, mas o código em si ainda está sendo executado por apenas uma thread, limitada pelo Global Interpreter Lock (GIL) do Python.

Machine Translated by Google

O código começa importando módulos essenciais de langchain_openai e langchain_core, incluindo componentes para modelos de linguagem, prompts, análise de saída e estruturas executáveis. O código tenta inicializar uma instância do ChatOpenAI, especificamente usando o modelo "gpt-4o-mini", com uma temperatura especificada para controlar a criatividade. Um bloco try-except é usado para garantir robustez durante a inicialização do modelo de linguagem. Três "cadeias" independentes do LangChain são então definidas, cada uma projetada para executar uma tarefa distinta no tópico de entrada. A primeira cadeia serve para resumir o tópico de forma concisa, usando uma mensagem do sistema e uma mensagem do usuário contendo o marcador de posição do tópico. A segunda cadeia é configurada para gerar três perguntas interessantes relacionadas ao tópico. A terceira cadeia é configurada para identificar entre 5 e 10 termos-chave do tópico de entrada, solicitando que sejam separados por vírgulas.
Cada uma dessas cadeias independentes consiste em um ChatPromptTemplate adaptado à sua tarefa específica, seguido pelo modelo de linguagem inicializado e um StrOutputParser para formatar a saída como uma string.

Um bloco RunnableParallel é então construído para agrupar essas três cadeias, permitindo que sejam executadas simultaneamente. Esse runnable paralelo também inclui um RunnablePassthrough para garantir que o tópico de entrada original esteja disponível para as etapas subsequentes. Um ChatPromptTemplate separado é definido para a etapa final de síntese, recebendo o resumo, as perguntas, os termos-chave e o tópico original como entrada para gerar uma resposta abrangente. A cadeia de processamento completa de ponta a ponta, denominada full_parallel_chain, é criada sequenciando o map_chain (o bloco paralelo) no prompt de síntese, seguido pelo modelo de linguagem e pelo analisador de saída. Uma função assíncrona run_parallel_example é fornecida para demonstrar como invocar essa full_parallel_chain. Essa função recebe o tópico como entrada e usa invoke para executar a cadeia assíncrona. Finalmente, o bloco padrão do Python `if __name__ == "__main__":` mostra como executar o `run_parallel_example` com um tópico de exemplo, neste caso, "A história da exploração espacial", usando `asyncio.run` para gerenciar a execução assíncrona.

Em essência, este código configura um fluxo de trabalho onde múltiplas chamadas LLM (para resumo, perguntas e termos) ocorrem simultaneamente para um determinado tópico, e seus resultados são então combinados por uma chamada LLM final. Isso demonstra a ideia central de paralelização em um fluxo de trabalho agentivo usando LangChain.

# Exemplo prático de código (Google ADK)

Ok, vamos agora nos concentrar em um exemplo concreto que ilustra esses conceitos dentro da estrutura do Google ADK. Vamos examinar como os elementos básicos do ADK, como

Machine Translated by Google

ParallelAgent e SequentialAgent podem ser aplicados para construir um fluxo de agentes que aproveita a execução simultânea para maior eficiência.

```
from google.adk.agents import LlmAgent, ParallelAgent, SequentialAgent from google.adk.tools
import google_search
GEMINI_MODEL="gemini-2.0-flash"


# --- 1. Definir Subagentes de Pesquisa (para execução em paralelo) ---

# Pesquisador 1: Energia Renovável researcher_agent_1
= LlmAgent(
        nome="PesquisadorDeEnergiasRenováveis",
        modelo=GEMINI_MODEL,
        instrução="""Você é um Assistente de Pesquisa em IA especializado em
energia.
Pesquise os avanços mais recentes em 'fontes de energia renováveis'.
Utilize a ferramenta de busca do Google fornecida.
Resuma suas principais conclusões de forma concisa (1-2 frases).
Exiba *apenas* o resumo. """,

        description="Pesquisa fontes de energia renováveis.", tools=[google_search], # Armazena
        o resultado no estado para o agente de
        fusão output_key="renewable_energy_result"

)

# Pesquisador 2: Veículos Elétricos researcher_agent_2 =
LlmAgent(
        nome="EVResearcher",
        modelo=GEMINI_MODEL,
        instrução="""Você é um Assistente de Pesquisa em IA especializado em transporte.

Pesquise os últimos avanços em 'tecnologia de veículos elétricos'.
Utilize a ferramenta de busca do Google fornecida.
Resuma suas principais conclusões de forma concisa (1-2 frases).
Exiba *apenas* o resumo. """,

        description="Pesquisa tecnologia de veículos elétricos.", tools=[google_search], # Armazena o
        resultado no estado para o agente de
        fusão output_key="ev_technology_result"

)

# Pesquisador 3: Captura de Carbono
researcher_agent_3 = LlmAgent(
```

Machine Translated by Google

```
        nome="CarbonCaptureResearcher", modelo=GEMINI_MODEL,
        instrução="""Você é um(a)
        Assistente de Pesquisa em IA especializado(a) em soluções climáticas.

Pesquise o estado atual dos 'métodos de captura de carbono'.
Utilize a ferramenta de busca do Google fornecida.
Resuma suas principais conclusões de forma concisa (1-2 frases).
Exiba *apenas* o resumo. """,

        description="Pesquisa métodos de captura de carbono.", tools=[google_search], # Armazena
        o resultado no estado para o agente de
        fusão output_key="carbon_capture_result"

)

# --- 2. Criar o ParallelAgent (Executa pesquisadores simultaneamente) --- # Este agente orquestra a execução simultânea dos
pesquisadores.

# O processo termina quando todos os pesquisadores concluírem e armazenarem seus resultados no estado.
parallel_research_agent =
ParallelAgent( name="ParallelWebResearchAgent",
        sub_agents=[researcher_agent_1, researcher_agent_2,
        researcher_agent_3], description="Executa vários agentes de pesquisa em paralelo para coletar


Informação." )


# --- 3. Defina o Agente de Fusão (Executa *após* os agentes paralelos)
---
# Este agente recebe os resultados armazenados no estado da sessão pelos agentes paralelos # e os sintetiza em uma única
resposta estruturada com
atribuições. merger_agent = LlmAgent(


        nome="AgenteDeSíntese",
        modelo=GEMINI_MODEL, # Ou potencialmente um modelo mais poderoso, se necessário para a síntese
instrução="""Você é um Assistente
        de IA responsável por combinar
Transformar os resultados da pesquisa em um relatório estruturado.
Sua principal tarefa é sintetizar os seguintes resumos de pesquisa, atribuindo claramente as descobertas às suas fontes originais.
Estruture sua resposta utilizando títulos para cada tópico. Certifique-se de que o relatório seja coerente e integre os pontos
principais de forma fluida.


**Crucialmente importante: Sua resposta DEVE ser baseada *exclusivamente* nas informações fornecidas nos 'Resumos de
Entrada' abaixo. NÃO adicione nada além disso.**
```

Machine Translated by Google

Qualquer conhecimento externo, fato ou detalhe não presente nestes resumos específicos.**

**Resumos de entrada:**

* **Energia renovável:**
{resultado_de_energia_renovável}
* **Veículos Elétricos:** {ev_technology_result}

* **Captura de carbono:**
{resultado_da_captura_de_carbono}

**Formato de saída:**

Resumo dos recentes avanços em tecnologia sustentável

### Resultados sobre Energias Renováveis
(Baseado nas descobertas do RenewableEnergyResearcher)
[Sintetize e elabore *apenas* com base no resumo de entrada de energia renovável fornecido acima.]

### Resultados sobre veículos elétricos (Baseado
nas descobertas da EVResearcher)
[Sintetize e elabore *apenas* com base no resumo de entrada do EV fornecido acima.]

### Resultados da Captura de Carbono
(Baseado nas descobertas do CarbonCaptureResearcher)
[Sintetize e elabore *apenas* com base no resumo dos insumos para captura de carbono fornecido acima.]

### Conclusão Geral [Forneça uma
breve declaração conclusiva (1-2 frases) que conecte *apenas* as descobertas apresentadas acima.]

Gere *apenas* o relatório estruturado seguindo este formato. Não inclua frases introdutórias ou conclusivas
fora desta estrutura e utilize estritamente apenas o conteúdo do resumo fornecido.

Combina resultados de pesquisa de agentes paralelos em
Um relatório estruturado e com citações, estritamente baseado nas informações fornecidas.",
# Nenhuma ferramenta necessária para
mesclar # Nenhuma chave de saída necessária aqui, pois sua resposta direta é o resultado final
saída da sequência)

# --- 4. Criar o SequentialAgent (Orquestra o fluxo geral)

Machine Translated by Google

```
---

# Este é o agente principal que será executado. Primeiro, ele executa o ParallelAgent # para preencher o estado e, em
seguida, executa o
MergerAgent para produzir a saída final. sequential_pipeline_agent = SequentialAgent( name="ResearchAndSynthesisPipeline", #
Executa a pesquisa paralela
primeiro e, em seguida, mescla sub_agents=[parallel_research_agent, merger_agent],
        description="Coordena a pesquisa paralela e sintetiza os resultados."





) agente_raiz = agente_de_pipeline_sequencial
```

Este código define um sistema multiagente usado para pesquisar e sintetizar informações sobre avanços em tecnologias sustentáveis. Ele configura três instâncias de LlmAgent para atuarem como pesquisadores especializados. O ResearcherAgent_1 concentra-se em fontes de energia renováveis, o ResearcherAgent_2 pesquisa tecnologia de veículos elétricos e o ResearcherAgent_3 investiga métodos de captura de carbono. Cada agente pesquisador é configurado para usar um GEMINI_MODEL e a ferramenta google_search. Eles são instruídos a resumir suas descobertas de forma concisa (1-2 frases) e armazenar esses resumos no estado da sessão usando output_key.

Em seguida, é criado um ParallelAgent chamado ParallelWebResearchAgent para executar esses três agentes de pesquisa simultaneamente. Isso permite que a pesquisa seja conduzida em paralelo, potencialmente economizando tempo. O ParallelAgent conclui sua execução assim que todos os seus subagentes (os pesquisadores) terminarem e preencherem o estado.

Em seguida, define-se um MergerAgent (também um LlmAgent) para sintetizar os resultados da pesquisa. Este agente recebe como entrada os resumos armazenados no estado da sessão pelos pesquisadores em paralelo. Suas instruções enfatizam que a saída deve ser estritamente baseada apenas nos resumos de entrada fornecidos, proibindo a adição de conhecimento externo. O MergerAgent foi projetado para estruturar as descobertas combinadas em um relatório com títulos para cada tópico e uma breve conclusão geral.

Finalmente, um SequentialAgent chamado ResearchAndSynthesisPipeline é criado para orquestrar todo o fluxo de trabalho. Como controlador principal, este agente principal executa primeiro o ParallelAgent para realizar a pesquisa. Assim que o ParallelAgent for concluído, o SequentialAgent executa o MergerAgent para sintetizar as informações coletadas. O sequential_pipeline_agent é definido como o root_agent, representando o ponto de entrada para executar este sistema multiagente. O processo geral

Machine Translated by Google

É projetado para coletar informações de múltiplas fontes em paralelo de forma eficiente e, em seguida, combiná-las em um único relatório estruturado.

# Em resumo

**O que:** Muitos fluxos de trabalho com agentes envolvem múltiplas subtarefas que devem ser concluídas para atingir um objetivo final. Uma execução puramente sequencial, onde cada tarefa espera a conclusão da anterior, costuma ser ineficiente e lenta. Essa latência se torna um gargalo significativo quando as tarefas dependem de operações de E/S externas, como chamadas a diferentes APIs ou consultas a múltiplos bancos de dados. Sem um mecanismo para execução concorrente, o tempo total de processamento é a soma da duração de todas as tarefas individuais, prejudicando o desempenho e a capacidade de resposta do sistema como um todo.

**Porquê:** O padrão de Paralelização oferece uma solução padronizada, permitindo a execução simultânea de tarefas independentes. Ele funciona identificando componentes de um fluxo de trabalho, como o uso de ferramentas ou chamadas de LLM (Language-Level Management), que não dependem das saídas imediatas uns dos outros. Frameworks agentivos como o LangChain e o Google ADK fornecem construções integradas para definir e gerenciar essas operações concorrentes. Por exemplo, um processo principal pode invocar várias subtarefas que são executadas em paralelo e aguardar a conclusão de todas elas antes de prosseguir para a próxima etapa. Ao executar essas tarefas independentes simultaneamente, em vez de uma após a outra, esse padrão reduz drasticamente o tempo total de execução.

**Regra geral:** Use esse padrão quando um fluxo de trabalho contiver várias operações independentes que podem ser executadas simultaneamente, como buscar dados de diversas APIs, processar diferentes blocos de dados ou gerar vários conteúdos para síntese posterior.

**Resumo visual**

Machine Translated by Google

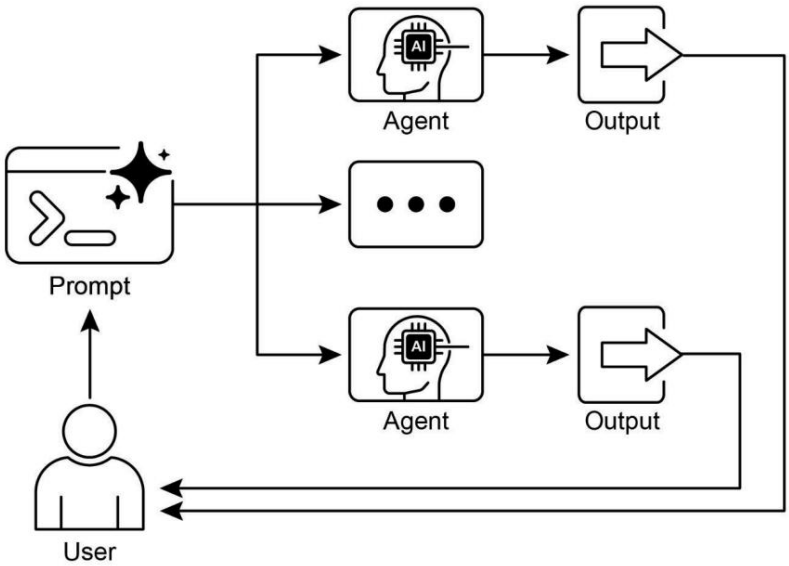


Figura 2: Padrão de projeto de paralelização

## Principais conclusões

Aqui estão os principais pontos a serem destacados:

- A paralelização é um padrão para executar tarefas independentes simultaneamente. Aumentar a eficiência.
- É particularmente útil quando as tarefas envolvem a espera por recursos externos, como como chamadas de API.
- A adoção de uma arquitetura concorrente ou paralela introduz complexidade e custos substanciais, impactando fases-chave do desenvolvimento, como projeto, depuração e registro de logs do sistema. • Frameworks como LangChain e Google ADK oferecem suporte integrado para Definição e gestão da execução paralela.
- Na Linguagem de Expressão LangChain (LCEL), RunnableParallel é uma construção fundamental. para executar vários executáveis lado a lado.

Machine Translated by Google

- O Google ADK pode facilitar a execução paralela por meio da delegação orientada por LLM. onde o LLM de um agente coordenador identifica subtarefas independentes e aciona seu processamento simultâneo por subagentes especializados.
- A paralelização ajuda a reduzir a latência geral e torna os sistemas agentes mais Responsivo para tarefas complexas.

# Conclusão

O padrão de paralelização é um método para otimizar fluxos de trabalho computacionais, executando subtarefas independentes simultaneamente. Essa abordagem reduz a latência geral, principalmente em operações complexas que envolvem múltiplas inferências de modelos ou chamadas a serviços externos.

Os frameworks oferecem mecanismos distintos para implementar esse padrão. No LangChain, construções como RunnableParallel são usadas para definir e executar explicitamente múltiplas cadeias de processamento simultaneamente. Em contraste, frameworks como o Google Agent Developer Kit (ADK) podem alcançar a paralelização por meio de delegação multiagente, onde um modelo coordenador primário atribui diferentes subtarefas a agentes especializados que podem operar concorrentemente.

Ao integrar o processamento paralelo com fluxos de controle sequenciais (encadeamento) e condicionais (roteamento), torna-se possível construir sistemas computacionais sofisticados e de alto desempenho, capazes de gerenciar com eficiência tarefas diversas e complexas.

# Referências

Aqui estão alguns recursos para leitura complementar sobre o padrão de paralelização e conceitos relacionados:

1. Documentação da Linguagem de Expressão LangChain (LCEL) (Paralelismo): https://python.langchain.com/docs/concepts/lcel/
2. Documentação do Google Agent Developer Kit (ADK) (Sistemas Multiagentes): https://google.github.io/adk-docs/agents/multi-agents/
3. Documentação do asyncio em Python: https://docs.python.org/3/library/asyncio.html